

Ultrasonic Distance Monitor with Relay-Controlled LED

COURSE PROJECT PROPOSAL

JAVA-ARDUINO USB SERIAL COMMUNICATION

This proposal presents a complete design for a Smart Distance Monitoring System built around the Arduino UNO and a Java Swing GUI. The system uses an HC-SR04 ultrasonic sensor to measure real-time distance in centimeters, automatically activates a warning LED via a relay when an object enters a configurable threshold zone, and displays live readings on a 1602 LCD. The Java GUI enables users to set distance thresholds, toggle between Auto and Manual modes, and manually control the LED — all over a standard USB serial connection. This document covers the project idea, component inventory, serial protocol design, wiring plan, GUI layout, and a justification of how every course requirement is satisfied.

Author: Dilan Janaka Hewarahinduwege | Date: June 14, 2026

Project Idea

The Ultrasonic Distance Monitor is a two-way serial communication project that bridges a Java Swing desktop application with an Arduino UNO microcontroller. At its core, the system continuously measures the distance to the nearest object using an HC-SR04 ultrasonic sensor and sends that data to the Java GUI over USB serial. The user can configure a distance threshold from the GUI — when an object moves closer than this threshold, the Arduino automatically triggers a relay to illuminate a warning LED. This provides a clear, physical response to sensor input without requiring any manual intervention in Auto mode.

Beyond the primary sensor and relay output, the project incorporates several additional inputs and outputs to demonstrate robust embedded systems design. A 1602 LCD (operating in 4-bit mode without an I2C backpack) displays live distance readings, the current threshold value, and system status directly on the hardware — giving the Arduino side its own independent feedback loop. A 10k Ω potentiometer adjusts LCD contrast in real time. A push button provides local manual override of the LED, and a slide switch toggles between Auto mode (threshold-driven) and Manual mode (GUI-driven LED control). An optional passive buzzer adds an audible alert when the threshold is exceeded.

The serial protocol is intentionally simple: line-based KEY:VALUE messages terminated with a newline character. This keeps parsing straightforward on both the Java and Arduino sides while remaining human-readable for debugging. The Arduino sends periodic telemetry (DIST:), acknowledgements (ACK:), and state-change events (LED:ON, MODE:AUTO). The Java GUI sends commands (THRESH:value, LED:ON, MODE:MANUAL) and receives confirmations for each. Every aspect of this design — from component selection to protocol format — was chosen to satisfy the course requirements while remaining buildable within a 4-week timeline and an estimated 12–16 hours of hands-on work.



Sensor Input

HC-SR04 ultrasonic sensor measures distance to nearest object in centimeters, updated every second.



Relay Output

5V relay switches a warning LED automatically when distance falls below the user-defined threshold.



Java GUI

Swing-based desktop app for serial control, threshold setting, mode selection, and live data display.



LCD Display

1602 LCD shows real-time distance, threshold, and status on the Arduino side independently.

Sensor Inputs & Visible Outputs

The project incorporates three distinct input types and three visible output types, exceeding the minimum course requirements. The primary sensor — the HC-SR04 — provides continuous distance measurements that drive the core logic of the system. Two additional user inputs (a push button and a slide switch) give the user local control over the Arduino without needing the GUI. On the output side, the relay-controlled LED is the primary visible effect, but the LCD and optional buzzer provide supplementary feedback that makes the system feel complete and polished during a live demonstration.

Type	Component	Purpose
Input — Sensor	HC-SR04 Ultrasonic	Measures distance (cm) to nearest object
Input — User	Push Button	Manual override / LED toggle (local control)
Input — User	Slide Switch (SPDT)	Toggle between Auto and Manual mode
Input — Analog	Potentiometer (10k Ω)	Adjust LCD contrast
Output — Visible	Relay + Red LED	Activates warning light when distance < threshold
Output — Visible	LCD 1602 (4-bit mode)	Displays live distance, threshold, and status
Output — Audible	Passive Buzzer (optional)	Audible alert when threshold exceeded

Auto Mode Behavior

In Auto mode, the Arduino independently monitors the HC-SR04 sensor. When the measured distance drops below the configured threshold, the relay activates and the LED turns on immediately. The LCD updates in real time, and an optional buzzer can sound. The Java GUI displays the current distance and LED state but does not intervene — the Arduino handles everything locally.

Manual Mode Behavior

In Manual mode, the Arduino still measures distance and sends telemetry to the GUI, but the relay is controlled exclusively by Java commands. The user clicks **LED:ON** or **LED:OFF** buttons in the GUI to toggle the warning light. The push button on the Arduino side can also toggle the LED locally, providing a fallback if the serial connection is unavailable.

Components & Availability

One of the strengths of this proposal is that every required component is already in the author's inventory. There are no procurement delays, no budget concerns, and no uncertainty about part compatibility. The Arduino UNO R3, HC-SR04 sensor, LCD 1602, relay module, and all supporting components (resistors, jumper wires, breadboard) are confirmed available. This means work can begin immediately upon approval, and the full 4-week timeline can be dedicated to assembly, programming, and testing rather than waiting for parts to arrive.



Arduino UNO R3

Main microcontroller. Drives all sensor reading, LCD output, relay control, and serial communication.
Libraries: `LiquidCrystal` and `NewPing`.



HC-SR04 Ultrasonic Sensor

Measures distance from 2 cm to 400 cm with 3 mm accuracy. Connected to pins 8 (TRIG) and 9 (ECHO). Readings sent every 1 second.



LCD 1602 (No I2C)

16×2 character display in 4-bit mode. Shows live distance, threshold value, and Auto/Manual status. Contrast adjusted via potentiometer on A0.



5V Relay Module (KS2E-M-DC5)

Opto-isolated relay module controlled via pin 12. Switches the warning LED circuit. NO1 terminal connects to LED anode through a 1kΩ current-limiting resistor.



All components confirmed in inventory. No additional purchases required. Optional passive buzzer (KS0019) is also available if an audible alert is desired for the live demonstration.

Draft Serial Protocol

The serial protocol is the backbone of communication between the Java GUI and the Arduino. It uses a simple, line-based KEY:VALUE format terminated with a newline character (\n), making it human-readable in any serial monitor and straightforward to parse on both ends. The protocol is divided into two message directions: Arduino → Java (telemetry and events) and Java → Arduino (commands). Every command sent by Java receives an acknowledgement (ACK:command) from the Arduino, ensuring the GUI always knows whether a command was received and applied.

Arduino → Java (Telemetry & Events)

Message	Example	Meaning	Frequency
DIST:value	DIST:45.2	Current distance in cm	Every 1 second
THRESH:value	THRESH:20	Current distance threshold	On change or request
LED:ON / LED:OFF	LED:ON	LED (relay) state change	On state change
MODE:AUTO / MODE:MANUAL	MODE:AUTO	System mode	On mode change
STATUS:READY	STATUS:READY	Arduino booted and ready	Once on startup
ACK:command	ACK:THRESH:15	Acknowledges Java command	On command receipt

Java → Arduino (Commands)

Command	Example	Meaning
THRESH:value	THRESH:15	Set distance threshold (range: 5–200 cm)
LED:ON / LED:OFF	LED:ON	Manual LED control (Manual mode only)
MODE:AUTO / MODE:MANUAL	MODE:MANUAL	Switch operating mode
REQUEST:DATA	REQUEST:DATA	Request immediate sensor reading
RESET	RESET	Reset threshold to default (30 cm)

Example Serial Session

```
// Arduino boots and sends status
STATUS:READY

// Arduino sends regular telemetry
DIST:45.2
LED:OFF
MODE:AUTO
```

Arduino UNO Component Connections

The wiring plan uses 11 digital pins and 1 analog pin on the Arduino UNO, leaving several pins free for future expansion. Power distribution is straightforward: the 5V rail supplies the HC-SR04, LCD VDD, potentiometer, relay module, and push button. All component grounds connect to the Arduino GND rail. The LCD operates in 4-bit mode without an I2C adapter, requiring six data/control pins (RS, E, D4–D7). The HC-SR04 uses two dedicated pins (TRIG on pin 8, ECHO on pin 9). The relay is controlled via pin 12, and the slide switch and push button occupy pins 11 and 10 respectively. The potentiometer wiper connects to A0 for optional contrast monitoring in software.

Pin Mapping Table

Arduino Pin	Component	Connection Purpose
2	LCD	RS — Register Select
3	LCD	E — Enable
4	LCD	D4 — Data bit 4
5	LCD	D5 — Data bit 5
6	LCD	D6 — Data bit 6
7	LCD	D7 — Data bit 7
8	HC-SR04	TRIG — Trigger pin
9	HC-SR04	ECHO — Echo pin
10	Push Button	Manual override input (pull-down)
11	Slide Switch	Mode selector (Auto=HIGH, Manual=LOW)
12	Relay Module	IN — Control signal (HIGH = LED ON)
A0	Potentiometer	LCD contrast (optional analog monitoring)

LCD 1602 Connections (4-bit Mode)

LCD Pin	Connected To
VSS	GND
VDD	5V
VO	Potentiometer wiper (contrast)
RS	Arduino Pin 2
RW	GND

Pin Summary

The Arduino UNO has 14 digital I/O pins and 6 analog input pins. This project uses 11 digital pins and 1 analog pin, which is well within the available capacity. The pin assignments were chosen to group related functions together (LCD on pins 2–7, sensor on pins 8–9, controls on pins 10–12) for clean wiring and logical code structure. Pin A0 is used for the potentiometer wiper, primarily for LCD contrast adjustment, though the analog value can also be read in software if needed for diagnostics.

Pins 2–7: LCD Data & Control

RS (pin 2), E (pin 3), and D4–D7 (pins 4–7) form the 4-bit interface to the LCD 1602. This is the largest pin block and uses the standard `LiquidCrystal` library pin order.

Pins 8–9: Ultrasonic Sensor

TRIG (pin 8) sends the pulse; ECHO (pin 9) receives the return. The `NewPing` library handles timing and distance calculation, returning values in centimeters.

Pins 10–12: User Controls & Relay

Push button (pin 10) for local LED toggle, slide switch (pin 11) for Auto/Manual mode, and relay control (pin 12) for LED switching. All three are digital input or output.

Pin A0: Potentiometer

Analog input for LCD contrast adjustment. The wiper of the 10k Ω potentiometer connects here. The Arduino can optionally read this value to detect contrast changes.

11

Digital Pins Used

Pins 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12

1

Analog Pin Used

Pin A0 for potentiometer

3

Pins Remaining

Digital pins 0, 1, 13 free (0/1 reserved for USB)

16

Total I/O Points

14 digital + 6 analog on Arduino UNO

GUI Design (Planned)

The Java Swing GUI is designed to be clean, functional, and informative. It provides full control over the Arduino system while displaying all incoming telemetry in real time. The layout is divided into logical zones: a connection bar at the top for serial port management, a central data display area showing distance and threshold, a control panel for mode and LED commands, and a scrollable event log at the bottom that records every system event with timestamps. The GUI uses standard Swing components — JComboBox for port selection, JSlider for threshold, JRadioButton for mode, and JTextArea for the event log — ensuring compatibility and straightforward implementation.

```
+-----+
|           Ultrasonic Distance Monitor           |
|           with Relay-Controlled LED             |
+-----+
| COM Port: [COM3 ▼] [Connect] [Disconnect]       |
| Status: ● Disconnected                         |
+-----+
| CURRENT DISTANCE: 45.2 cm                       |
| [|||||----] 45.2 / 100 cm                      |
| THRESHOLD: [====o====] 30 cm [Set]             |
| LED STATUS: ● OFF                               |
| MODE: (●) Auto  ( ) Manual                     |
| MANUAL CONTROLS:                               |
| [TURN LED ON]  [TURN LED OFF]                 |
+-----+
| EVENT LOG:                                     |
| [12:30:01] Connected to COM3                   |
| [12:30:02] Mode: Auto, Threshold: 30cm         |
| [12:30:05] Threshold set to 15cm               |
| [12:30:10] Distance: 12cm - LED turned ON      |
| [12:30:15] Distance: 25cm - LED turned OFF    |
+-----+
```

GUI Controls

- **COM Port** — Dropdown selector for serial port
- **Connect/ Disconnect** — Manage USB serial connection
- **Mode Radio Buttons** — Auto / Manual mode selection
- **LED ON / OFF Buttons** — Manual LED control (Manual mode only)
- **Threshold Slider** — Set distance threshold (5–200 cm)
- **Set Threshold Button** — Apply slider value to

Displayed Data (Real-time)

- **Distance (cm)** — Numeric display + progress bar
- **Threshold (cm)** — Numeric display + slider position
- **LED Status** — Indicator light + ON/OFF text
- **Mode** — Highlighted radio button (Auto or Manual)
- **Connection Status** — Colored indicator (green/red)
- **Event Log** — Timestamped scrollable text area

All data elements update dynamically as serial messages arrive from the Arduino. The progress bar visually represents distance relative to a 100 cm scale for quick at-a-glance

What Makes This Project Complete

The course requirements for the Java-Arduino USB Serial Communication project specify a set of minimum capabilities that any submission must demonstrate. This proposal addresses every requirement explicitly and, in several cases, exceeds the baseline. The table below maps each requirement to the specific design element that satisfies it, providing a clear audit trail for the instructor and peers reviewing this proposal.

Requirement	How It Is Met
Java Swing GUI sends commands	Threshold set, mode switch, and manual LED control commands sent over serial
Arduino responds (ACK/events)	ACK:command + real sensor data + state change events (LED:ON, MODE:MANUAL)
Visible physical effect	LED turns on/off via relay; LCD updates in real time; optional buzzer sounds
At least one sensor reading	HC-SR04 ultrasonic sensor provides distance in cm, updated every second
Arduino sends telemetry	Periodic DIST: messages + STATUS:READY + state change events
GUI displays incoming data	Real-time numbers, progress bar, LED indicator, and scrollable event log
Connect/disconnect handling	COM port dropdown + Connect/Disconnect buttons + safe serial close
Error handling	Invalid threshold range, missing sensor detection, serial timeout handling



Two-Way Serial

Full bidirectional communication: Java commands → Arduino ACKs + telemetry → Java display.



Three Visible Effects

Relay LED, LCD display, and optional buzzer — three physical outputs from one sensor input.



Six GUI Controls

COM selector, Connect, Disconnect, Mode radios, Threshold slider, Set button, LED buttons.



4-Week Timeline

Estimated 12–16 hours distributed across wiring, Arduino code, Java GUI, testing, and docs.

Conclusion

The Ultrasonic Distance Monitor with Relay-Controlled LED is a well-scoped, fully feasible project that satisfies every requirement of the Java-Arduino USB Serial Communication course project. The design is grounded in real, available hardware, uses standard Arduino libraries (`LiquidCrystal` for the LCD and `NewPing` for the ultrasonic sensor), and implements a clean, documented serial protocol that is easy to parse on both the Java and Arduino sides. The estimated development effort of 12–16 hours is realistic and can be distributed across wiring, Arduino programming, Java Swing development, integration testing, and final documentation within the 4-week project timeline.

Two-way serial communication is the centerpiece of this project. The Java Swing GUI sends structured commands — threshold updates, mode switches, and manual LED control — and the Arduino responds with acknowledgements, periodic distance telemetry, and state-change events. The protocol format (`KEY:VALUE\n`) is simple enough to debug in any serial monitor yet robust enough for reliable operation. The GUI features at least six interactive controls, a real-time progress bar for sensor data, and a timestamped event log that records every system action.

On the hardware side, three visible effects (relay-controlled LED, live LCD updates, and an optional buzzer) demonstrate that the Arduino is not just sending data but also acting on it autonomously. The HC-SR04 sensor provides the primary input, while the push button and slide switch add local control depth. All components are confirmed in inventory, eliminating procurement risk. This M1 Proposal is ready for instructor approval — the design is clear, straightforward to build on real hardware, and well-suited for a successful live defense.



Ready for Approval. This proposal meets all course requirements, all components are available, and the implementation plan is clear. Prepared by **Dilan Janaka Hewarahinduwage** for the Java-Arduino USB Serial Communication course, June 14, 2026.